

Lecture 11

1. The Reducer Concept

A **Reducer** is a functional programming concept. It is a "pure" function that takes the **previous state** and an **action**, and returns a **new state**.

The formula looks like this:

(State, Action) => NewState

Key Components:

- **Initial State:** The starting value of your data.
- **Action:** An object that describes what happened (usually has a `type` and an optional `payload`).
- **Reducer Function:** The logic that determines how the state changes based on the action.
- **Dispatch:** The function you call to "send" an action to the reducer.

2. `useReducer` in React

The `useReducer` hook is an alternative to `useState` for complex state logic. It is especially useful when the next state depends on the previous one or when you have multiple sub-values.

Syntax:

JavaScript

```
const [state, dispatch] = useReducer(reducer, initialState);
```

Simple Counter Example:

JavaScript

```
function reducer(state, action) {  
  switch (action.type) {  
    case 'increment':
```

```
    return { count: state.count + 1 };  
  case 'decrement':  
    return { count: state.count - 1 };  
  default:  
    throw new Error();  
  }  
}
```

3. useState vs. useReducer

Think of `useState` as a light switch (on/off) and `useReducer` as a complex dashboard with many buttons and dials.

Feature	useState	useReducer
State Complexity	Simple (strings, numbers, booleans)	Complex (nested objects, arrays)
Logic Location	Inside the component/event handlers	Moved to a separate reducer function
Predictability	Easy to follow for basic transitions	Better for complex state transitions
Testing	Harder to test state logic in isolation	Easy to test the reducer function separately

Performance	Standard	Can optimize performance for deep updates
-------------	----------	---

When to use which?

- **Use `useState` when:** You have independent pieces of state (like `isSelected` or `userName`) or simple data that doesn't rely on other state pieces.
- **Use `useReducer` when:** You have a "state object" where updating one part depends on another, or if your state transitions are getting messy with too many `useState` calls. It keeps your component clean by moving the "how it changes" logic elsewhere.

Reducer javascript :

```
const actions = [ //action
  { type: 'DEPOSIT', amount: 100 },
  { type: 'WITHDRAW', amount: 50 },
  { type: 'DEPOSIT', amount: 20 }
];

const initialState = { balance: 0 }; //state

const accountReducer = (state, action) => {
  if (action.type === 'DEPOSIT') return { balance: state.balance +
    action.amount };
  if (action.type === 'WITHDRAW') return { balance: state.balance -
    action.amount };
  return state;
};

// The reducer processes the actions one by one
const finalBalance = actions.reduce(accountReducer, initialState);
```

```
console.log(finalBalance); // { balance: 70 }
```

```
//Reducer function will always take two arguments (state, action)  
//reduce function will take (reducer function , intial state)  
//Old state ---> New State  
//
```

useReducer :

```
// state → current state value
```

```
// dispatch → function used to send actions
```

```
// reducer → function that decides how state updates
```

```
// initialState → starting value
```

```
//Reducer function will always take two arguments (state, action)  
//reduce function will take (reducer function , intial state)  
//Old state ---> New State
```

```
import React, { useReducer, useState } from "react";
```

```
function reducer(state, action) {  
  switch (action.type) {  
    case "INCREMENT":  
      return { count: state.count + 1 };  
  
    case "DECREMENT":  
      return { count: state.count - 1 };  
  
    default:  
      return state;  
  }  
}
```

```
//useReducer === reduce keyword
```

```
export default function Counter() {

  const [count, setCount]=useState(0)

  const [state, dispatch] = useReducer(reducer, { count: 0 });

  return (
    <div>
      <h1>{state.count}</h1>

      <button onClick={() => dispatch({ type: "INCREMENT" })}>+</button>

      <button onClick={() => dispatch({ type: "DECREMENT" })}>-</button>
    </div>
  );
}
```